When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

1. Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

2. Design/Methodology/Approach

This is a design science study. We read the code of an existing risk management application, ran it, and recorded seven distinct ways in which it failed. Each failure was traced to theory that already explains it, from integrity models, fail-safe defaults and complete mediation to separation of duties, tamper-evident logging and the critique of audit as ritual, and a design principle was drawn from that theory. We built a new system, ERMA-GRC, on these principles. The old and new systems then faced 23 scenarios over the same stored data, under a protocol time-stamped in the project repository beforehand. Eighteen scenarios come from the audit, four go beyond it, and one was meant to fail. All deviations are reported.

3. Findings

There are 18 cases created from the audit; the old system prevented the failure in 4 of these. ERMA-GRC prevented all 18. ERMA-GRC prevents write actions outside sessions, stops users approving their own records, stamps audit trail lines with the logged-in user rather than a claimed name, detects edits to an entry and indicates unavailability of the database as a failure. Beyond the audit cases, ERMA-GRC prevented 3 out of 4 failures, including two simultaneous approvals for one record. Results come from the database stand-in, not from what either system reported. The new design failed in two cases. Although a key risk indicator shows the correct state on the screen, a client can still connect to the server and store any state it likes. Also, a person with the audit key can manipulate the audit trail offline without being caught once the server restarts. Principles depend on each other because separation of duties depends on the reliability of the audit trail.

This Abstract is submitted to LEAD-2027 conference conducted at IIM Shillong during 20-22 January 2027

4. Originality/Value

We treat the integrity of assurance evidence as something a GRC system's design can protect or destroy, rather than as an organisational outcome. This paper provides a taxonomy of failures observed in a working system, seven principles together with two testable propositions towards a design theory, and a scenario protocol that others can rerun. Buyers and auditors may utilize these principles in the form of a checklist. These two design problems lead to three further tasks: external anchoring of the audit trail outside the system; calculation of all metrics on the server; and the test of the principles with practitioners.

Keywords

assurance integrity; GRC information systems; design science research; enterprise risk management; tamper-evident audit trail; separation of duties